

PT-P2:Neural Network Testing

Report

Iverson Labastida
Dept. of Information Technology (BSIT)
Jose Rizal University
Mandaluyong City, Philippines
iverson.labastida@my.jru.edu

Abstract—This study investigates the impact of explicit code-based hyperparameter optimization, dataset volume scaling, and regularization techniques on Feedforward Backpropagation (FFBP) neural networks using TensorFlow/Keras, compared against a no-code automated baseline (PT-P1). Twenty experimental runs (EXP-001 through EXP-020) were executed on a ride-hailing text dataset. Initial experiments on a small dataset (35 rows, EXP-001 to EXP-007) revealed severe overfitting (training loss 0.0291 vs. validation loss 1.5693, validation accuracy capped at 20.00%), demonstrating that architectural capacity reduction alone cannot overcome sample size limits. Expanding the dataset to 500 rows (EXP-008 to EXP-014) yielded a dramatic performance increase, achieving 86.00% validation accuracy and 0.8698 F1-score. Controlled epoch scaling (EXP-015 to EXP-018) identified an operational plateau at 60 epochs before overfitting re-emerged at 70 epochs. Applying a dropout rate of 0.30 in EXP-020 eliminated the generalization gap, resulting in a smooth convergence with validation loss of 0.3586 and F1-score of 0.8275. While the no-code baseline achieved 90.80% accuracy via automated heuristics, code-based FFBP provides fine-grained parameter control essential for diagnosing loss dynamics and preventing model memorization.

Index Terms—Feedforward Backpropagation, Hyperparameter Optimization, Overfitting, Loss Curves, Regularization, Keras, Pipeline Mapping.

I. INTRODUCTION AND PT-P1 BASELINE PERFORMANCE

The rapid democratization of machine learning has introduced automated no-code development platforms, such as the PT-P1 baseline tool, which enable rapid model deployment without requiring manual programming. In initial evaluation, the PT-P1 no-code baseline tool achieved an impressive classification Accuracy of 90.80% and an F1-Score of 88.61%. However, no-code solutions inherently operate as 'black-box' systems: they rely on automated heuristics that abstract away internal loss convergence, backpropagation gradient dynamics, and structural regularization decisions.

While high baseline metrics provide an encouraging starting point, real-world deployment demands rigorous diagnostic control over neural network behavior. Without visibility into training versus validation loss curves, practitioners cannot determine whether a model has truly generalized or merely memorized subtle dataset artifacts. Explicit code-based development using deep learning frameworks such as TensorFlow and Keras restores full programmatic authority over hyperparameter selection, network architecture, gradient update algorithms, and regularization strategies.

This paper presents a comprehensive empirical investigation into Feedforward Backpropagation (FFBP) neural networks across twenty structured experimental runs (EXP-001 through EXP-020), recorded in the Labastida-FFBP Training Log. We systematically analyze the interplay between hyperparameter tuning, dataset volume expansion, epoch plateau dynamics, and dropout regularization. Furthermore, we provide a detailed line-by-line mapping between physical Keras code executions in the Iverson_Neural_Network_Training_Testing.ipynb notebook and theoretical deep learning pipeline architecture.

II. EXPERIMENTAL METHODOLOGY AND HYPERPARAMETER CONFIGURATIONS

A. Dataset Preprocessing and Partitioning

The empirical evaluation was conducted using two iterations of a ride-hailing text classification dataset: an initial small-scale dataset containing 35 rows (ride_hailing_dataset.csv) and an expanded dataset containing 500 rows (ride-hailing-dataset-expanded.csv). As implemented in Cell 8 of the notebook, raw text strings were preprocessed using a custom vocabulary builder (build_vocab) with a maximum vocabulary size MAX_VOCAB_SIZE = 300 and a fixed sequence length MAX_LEN = 15 tokens. Short sequences were padded with zero tokens (<PAD>), while longer sequences were truncated.

To evaluate model generalization accurately, the dataset was partitioned into 70% training (X_train, y_train), 15% validation (X_val, y_val), and 15% test (X_test, y_test) splits using train_test_split with random_state = 42 for complete experimental reproducibility.

B. Feedforward Backpropagation Neural Network Architecture

The underlying neural network was constructed using the Keras Sequential API (Cell 14). The model architecture comprises:

- 1) Input Embedding Layer: Embedding(input_dim=len(vocab), output_dim=embed_dim, mask_zero=True), mapping discrete token indices into dense continuous vector representations.
- 2) Spatial Feature Reduction Layer: GlobalAveragePooling1D(), averaging token vectors across the temporal length dimension.
- 3) Hidden Representation Layer: Dense(hidden_dim, activation='relu'), computing non-linear feature combinations.

- 4) Regularization Layer: Dropout(dropout_rate), randomly deactivating neurons during training to enforce feature redundancy.
- 5) Classification Layer: Dense(num_classes, activation='softmax'), emitting multi-class posterior probability distributions.

C. Summary of Experimental Configurations

TABLE I summarizes key representative configurations and metrics across the three experimental phases. (All 20 runs EXP-001 to EXP-020 are analyzed in Sections III and IV).

TABLE I. TABLE I. REPRESENTATIVE HYPERPARAMETER CONFIGURATIONS & RESULTS

Run ID	N	Hidden, LR, Drop	Ep.	Val Loss	Val Acc	Outcome
EXP-01	35	H=64, $\eta=0.005$, $p=.1$	15	1.0781	20.0%	Baseline
EXP-02	35	H=64, $\eta=0.005$, $p=.1$	30	1.5693	20.0%	Overfit
EXP-05	35	H=24, $\eta=0.0001$, $p=.1$	30	1.0945	60.0%	Underfit
EXP-08	500	H=24, $\eta=0.001$, $p=.1$	30	0.3180	85.0%	Expanded
EXP-12	500	H=12, $\eta=0.0007$, $p=.1$	30	0.3592	86.0%	Tuned LR
EXP-14	500	H=12, $\eta=0.0003$, $p=.1$	30	0.7154	86.0%	Low LR
EXP-17	500	H=12, $\eta=0.0003$, $p=.1$	60	0.3646	85.0%	Plateau
EXP-18	500	H=12, $\eta=0.0003$, $p=.1$	70	0.3738	85.0%	Overfit
EXP-20	500	H=12, $\eta=0.0003$, $p=.3$	70	0.3586	82.0%	Optimal

^a. a. Complete logs for all 20 runs (EXP-001–EXP-020) recorded in Labastida-FFBP Training Log.

III. PERFORMANCE MONITORING AND LOSS CURVE ANALYSIS

A. Quantitative Diagnostics of Loss Regimes

Loss curve trajectories serve as the principal diagnostic mechanism for evaluating neural network optimization dynamics. Three distinct regimes were identified across the 20 experimental runs:

1) **Overfitting Regime:** Characterized by a continuous decay in training loss ($L_{train} \rightarrow 0$) accompanied by a plateau or upward divergence in validation loss (L_{val}), yielding a substantial generalization gap $\Delta = L_{val} - L_{train}$. In EXP-002 (small dataset, $N=35$), extending training from 15 to 30 epochs collapsed training loss to 0.0291 while validation loss exploded to 1.5693, producing a massive generalization gap $\Delta = 1.5402$ and capping validation accuracy at 20.00%. In EXP-018 (expanded dataset, $N=500$), pushing training to 70 epochs caused validation loss to turn upward from 0.3646 (EXP-017) to 0.3738, signaling the onset of memorization despite high training accuracy (100%).

2) **Underfitting Regime:** Characterized by elevated training loss and validation loss where the network fails to extract discriminative features. In EXP-005, aggressively dropping the learning rate to 0.0001 restricted parameter updates, resulting in high training loss (1.0889) and validation loss (1.0945). In EXP-014 ($LR = 0.0003$), training loss remained high at 0.6196 with validation loss at 0.7154,

confirming that excessively low learning rates retard parameter convergence.

3) **Optimal Fit Regime:** Characterized by synchronous convergence of training and validation loss curves with a minimal, non-divergent generalization gap. In EXP-020 ($N=500$, hidden_dim=12, $LR=0.0003$, epochs=70, dropout=0.30), training loss converged cleanly to 0.2238 while validation loss stabilized at 0.3586 ($\Delta = 0.1348$), eliminating curve noise and preventing late-epoch divergence.

IV. DISCUSSION AND DEBRIEF QUESTIONS

A. Code-Based Optimization vs. No-Code Heuristics (PT-P1 Comparison)

Question 1 Statement: How did directly manipulating loss functions, learning rates, and gradient updates in Python (Keras) differ from relying on automated heuristics of the PT-P1 no-code tool? What trade-offs exist?

The PT-P1 no-code baseline tool demonstrated remarkable out-of-the-box efficacy, obtaining an Accuracy of 90.80% and an F1-Score of 88.61%. This high baseline is achieved through automated heuristics that optimize hyperparameter configurations behind a simplified user interface. However, our empirical findings reveal fundamental trade-offs between automated black-box heuristics and explicit Python code control:

- 1) **Diagnostic Visibility vs. Automated Speed:** No-code platforms hide batch-level loss trajectories and gradient norm updates. While PT-P1 produced a strong accuracy figure, it provided no diagnostic feedback when model performance stalled. In contrast, code-based Keras development enabled real-time loss tracking (Cell 18), allowing us to pinpoint the exact epoch (Epoch 60 in EXP-017) where validation loss plateaued before divergence.
- 2) **Fine-Grained Optimization Authority:** Python control allowed precise manipulation of optimization primitives: selecting between Adam and SGD optimizers (Cell 16), tuning learning rates across two orders of magnitude (0.0050 to 0.0003), configuring categorical loss formulations, and applying layer-specific dropout. When EXP-002 suffered severe overfitting (val loss 1.5693), no-code heuristics offered no parameter knobs for correction; explicit coding allowed us to introduce dropout regularization (EXP-019/EXP-020) to enforce generalization.
- 3) **Engineering Trade-Off:** No-code automation is optimal for rapid feasibility testing and non-technical stakeholder demonstration. Code-based feedforward backpropagation is essential for production machine learning, where custom data pipelines, loss curve debugging, and architectural regularization are required to prevent catastrophic failure on out-of-distribution inputs.

B. Diagnosing Model Fit via Loss Curves & Regularization (Pillar 1 & Pillar 3)

Question 2 Statement: What specific behavior in training vs. validation loss curves indicated Overfitting vs. Underfitting across EXP-01 to EXP-05? How did dropout_rate and hidden_dim adjustments resolve or exacerbate these issues?

Experiments EXP-001 through EXP-007 explicitly illustrated the structural limits of Architectural Capacity (Pillar 1) and Regularization (Pillar 3) when operating under data volume constraints ($N=35$):

1) Overfitting Diagnosis in EXP-001 to EXP-004: In EXP-001 (baseline, 15 epochs, hidden_dim=64), training loss reached 0.6152 while validation loss diverged to 1.0781. Extending training to 30 epochs in EXP-002 caused the network to completely memorize the 35 sample rows: training loss plummeted to 0.0291 while validation loss exploded to 1.5693 (generalization gap $\Delta = 1.5402$). Attempting to resolve this overfitting by reducing architectural capacity (hidden_dim reduced from 64 to 32 in EXP-003, and to 24 in EXP-004) slightly elevated training loss (0.0828 and 0.1175), but failed to cure overfitting because validation loss remained severely elevated (1.4220 and 1.4849).

2) Underfitting Induction in EXP-005: To force the overfitted network into an underfitting regime, EXP-005 dropped the learning rate to 0.0001. Training loss spiked to 1.0889 and validation loss reached 1.0945, demonstrating that when learning rates are insufficient, gradient backpropagation steps become too small to reach optimal loss minima within 30 epochs.

3) Regularization Efficacy in EXP-019 & EXP-020: Once dataset volume was expanded ($N=500$), extending epochs to 70 in EXP-018 re-introduced mild overfitting (val loss rising to 0.3738). Increasing dropout_rate from 0.10 to 0.20 (EXP-019) and 0.30 (EXP-020) successfully suppressed co-adaptation of hidden nodes, driving validation loss down to its lowest stable point (0.3586) and establishing a clean, regularized fit.

C. Data Volume & Diversity vs. Architectural Capacity

Question 3 Statement: Why must neural network capacity be balanced with dataset volume and representative diversity to prevent memorization?

A fundamental revelation of this experimentation is that architectural hyperparameter tuning cannot compensate for inadequate training data volume. In EXP-001 through EXP-007, the network possessed thousands of trainable parameters (Embedding weights plus Dense weights) but was trained on only 35 text samples. Because model capacity vastly exceeded dataset entropy, backpropagation optimization naturally drove parameter weights toward exact memorization of the 35 training sentences.

Expanding the dataset from 35 rows (ride_hailing_dataset.csv) to 500 rows (ride-hailing-dataset-expanded.csv) in EXP-008 fundamentally altered model behavior:

1) Validation Accuracy jumped from 20.00% (EXP-001 to EXP-004) to 85.00% (EXP-008).

2) Validation F1-Score increased dramatically from 0.2667 to 0.8544.

3) The generalization loss gap collapsed from 1.5402 (EXP-002) down to 0.2942 (EXP-008).

This empirical shift proves that neural network capacity (hidden_dim) and sample volume (N) exist in strict mathematical balance. High-capacity networks exposed to small, non-diverse datasets inevitably suffer memorization. Broadening data volume provides the representative feature distribution necessary for embedding layers to learn meaningful semantic boundaries rather than memorizing noise.

D. Pipeline Mapping: Connecting Code Implementation to Lecture Architecture

Question 4 Statement: Where exactly in Iverson Neural Network Training_Testing.ipynb do Forward Pass / Loss Calculation and Backward Pass / Weight Updates take place? How do Embedding and GlobalAveragePooling1D perform feature extraction?

To bridge physical code implementation with deep learning theoretical architecture (ITC 508 Lecture 3), we map the specific script cells and line numbers of the notebook to operational execution phases:

1) Automated Feature Extraction Phase (Cell 14, Lines 260–271):

- Embedding Layer (Lines 260–268): Converts sparse, tokenized integer sequences x_i in $\{0, \dots, 299\}$ into dense continuous vectors e_i in $\mathbb{R}^{16}$ using an embedding matrix W_{emb} in $\mathbb{R}^{(300 \times 16)}$.
- GlobalAveragePooling1D Layer (Line 271): Performs spatial dimension reduction across the temporal sequence ($\text{MAX_LEN} = 15$). It computes the mean embedding vector $e_{\text{avg}} = (1/T) \sum_{t=1}^T e_t$, producing a fixed 16-dimensional feature representation independent of text length prior to classification.

2) Forward Pass & Prediction Phase (Cell 14, Lines 274–280):

- Dense Hidden Layer (Line 274): Computes non-linear hidden activation $h = \text{ReLU}(W_1 e_{\text{avg}} + b_1)$, transforming features into a hidden dimension space (hidden_dim = 12).
- Dropout Layer (Line 277): Applies Bernoulli mask $r \sim \text{Bernoulli}(1 - p)$, setting hidden units to zero during training with probability $p = 0.35$.
- Dense Softmax Output Layer (Line 280): Computes class probability distribution $y_{\text{hat}} = \text{Softmax}(W_2 h + b_2)$.

3) Loss Computation Phase (Cell 16, Lines 308–313):

- Keras model.compile() configures sparse_categorical_crossentropy loss: $L = -\sum_k (y_k \log(y_{\text{hat}}_k))$, measuring categorical error between true label y and predicted probability y_{hat} .

4) Backward Pass & Weight Update Phase (Cell 18, Lines 340–347):

- Keras model.fit() executes the iterative backpropagation engine. For each mini-batch (batch_size = 64), TensorFlow executes automatic differentiation (Gradient Tape) to compute analytical loss gradients with respect to all trainable parameters: dL/dW_1 , dL/dW_2 , and dL/dW_{emb} .
- The Adam optimizer engine updates weights using moving average moments: $W^{(t+1)} = W^{(t)} - \eta * (m_t / (\sqrt{v_t} + \epsilon))$, adjusting internal model parameters to minimize loss across epochs.

V. CONCLUSION AND FUTURE WORK

This investigation demonstrates the critical necessity of explicit code-based hyperparameter tuning, data volume scaling, and regularization in feedforward backpropagation neural networks. Through 20 structured experimental runs on ride-hailing text classification data, we demonstrated that:

- 1) Data volume expansion from 35 to 500 rows was the single most decisive factor in eliminating severe overfitting, elevating validation accuracy from 20.00% to 86.00%.
- 2) Architectural capacity reduction (`hidden_dim=12`) combined with fine-grained learning rate tuning ($\eta = 0.0003$) stabilized loss convergence curves.
- 3) Introducing dropout regularization ($p = 0.30$) in EXP-020 eliminated late-epoch validation loss divergence, producing a clean, robust model fit.
- 4) While the PT-P1 no-code baseline tool offers impressive initial accuracy (90.80%), code-based TensorFlow/Keras development provides indispensable diagnostic visibility and architectural control required for production machine learning.

Future work will explore dynamic learning rate schedulers (e.g., Cosine Annealing), bidirectional recurrent layers (BiLSTM), and transformer-based self-attention embeddings to further enhance generalization bounds.

REFERENCES

- [1] [1] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.
- [2] [2] F. Chollet, Deep Learning with Python, 2nd ed. Shelter Island, NY, USA: Manning Publications, 2021.
- [3] [3] D. P. Kingma and J. Ba, 'Adam: A method for stochastic optimization,' in Proc. 3rd Int. Conf. Learn. Represent. (ICLR), San Diego, CA, USA, 2015, pp. 1–15.
- [4] [4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, 'Dropout: A simple way to prevent neural networks from overfitting,' J. Mach. Learn. Res., vol. 15, no. 1, pp. 1929–1958, 2014.
- [5] [5] I. Labastida, 'Feedforward Backpropagation Training Log and Notebook Evaluation,' Jose Rizal University, Department of BSIT, Tech. Rep. EXP-2026-FFBP, Aug. 2026.